

Webページから情報を取り出してみよう

はじめてのプログラミング / スクレイピング体験

きょうのゴール

Webページに並んでいる **30個の商品の名前と値段** を
プログラムに読ませて、**一瞬で** 全部並べます。

そのあと、**安い順にならべたり、平均を出したり** します。

手で写したら10分。プログラムなら1秒。

覚えることは、たった3つ

	やること	使うもの
1	取ってくる	requests
2	分解する	BeautifulSoup
3	探す	find / find_all

これだけです。文法は覚えなくて大丈夫。

「動いた」を積み重ねるのが今日の目的です。

きょうの道具

- **Google Colaboratory (Colab)** ブラウザだけで Python が動く。インストール不要
- **requests** ページを取ってくる係
- **BeautifulSoup** 取ってきたページをバラバラに分解して、探しやすいようにする係

準備はゼロ。ノートブックを開くだけです。

第1部

Webページの正体

ブラウザが見せているもの

みんながいつも見ているのは、こういう画面ですね。

- 見出しがあって
- 写真があって
- 文章があって
- 押すと飛ぶリンクがある

でも、コンピュータはこれを「絵」として持っているわけではありません。

正体は、ぜんぶ「文字」

Webページは、**全部が文字で書かれた設計図**です。

ブラウザは、その設計図を読んで
「じゃあこう表示しよう」と絵にしてくれているだけ。

その設計図の名前が HTML です。

HTML を見てみると

```
<h1>サイバー購買部</h1>  
<p>きょうの商品ラインナップ</p>
```

これがブラウザではこう見えます。

```
# サイバー購買部  
きょうの商品ラインナップ
```

同じものです。書き方が違うだけ。

タグ＝「ここは○○ですよ」という印

<h1>サイバー購買部</h1>

部分	名前	意味
<h1>	開始タグ	ここから見出しです
サイバー購買部	中身	実際に表示される文字
</h1>	終了タグ	見出しはここまで

< > で囲まれたものが **タグ**。閉じるほうには **/** が付きます。

タグの中に、タグが入る

```
<div>  
  <h2>メロンパン</h2>  
  <p>150円</p>  
</div>
```

同じ色が、**開きタグと閉じタグのペア**です。左の縦線が1組の範囲。

箱の中に、箱が入っているイメージです。

これを **入れ子（いれこ）** と呼びます。

今日はこの「箱」を目印にして、ほしいものを探します。

きょう出てくるタグ ①

タグ	読み	何を表すか
<title>	タイトル	ブラウザのタブに出る名前
<h1>	エイチワン	ページで一番大きい見出し
<h2>	エイチツー	その次の見出し

h は **heading（見出し）** の h。
数字が小さいほど大きい見出しです。

<title> と <h1> はちがう

	出る場所	購買部サイトでは
<title>	ブラウザのタブ	サイバー購買部 商品一覧
<h1>	ページの中	サイバー購買部

同じページなのに、中身が違います。

あとで実際に両方取って、違いを確かめます。

きょう出てくるタグ ②

タグ	読み	何を表すか
<p>	ピー	段落。ふつうの文章
<div>	ディブ	ただの箱。まとめるための入れもの
	スパン	文の中の一部を囲む小さい印

p は **paragraph**（段落）。

div は **division**（区切り）。

<div> は「箱」

```
<div>
  <h2>メロンパン</h2>
  <p>150円</p>
</div>
<div>
  <h2>焼きそばパン</h2>
  <p>180円</p>
</div>
```

div 自体は何も表示しません。

「ここからここまでが1セット」とまとめるだけです。

商品1つ分 = div 1つ。今日いちばん大事な考え方です。

＜span＞ は「文の中の一部」

```
<p><span>150</span>円</p>
```

ブラウザには **150円** と出ます。

でも ＜span＞ で囲んであるおかげで、

「150 の部分だけ」を狙って取り出せるようになります。

あとで効いてきます。覚えておいてください。

きょう出てくるタグ ③

```
<a href="items/item01.html">くわしく見る</a>
```

<a> は **リンク** (anchor=いかり、の a) 。

部分	意味
くわしく見る	画面に出る文字
href="..."	飛び先のアドレス

href のような、タグに付ける追加情報を **属性** といいます。

class = 名札

```
<h2 class="item-name">メロンパン</h2>
```

<h2> はページ中にたくさんあります。
そのままだと「どの h2 ? 」と区別が付きません。

そこで **名札 (class)** を付けておきます。

「item-name という名札の付いた h2 を全部持ってきて」
と頼めるようになります。

商品1つ分の中身（これが今日の地図）

```
<div class="item">
  <p class="item-icon">🍈</p>
  <h2 class="item-name">メロンパン</h2>
  <p class="item-category">パン</p>
  <p class="item-price-line"><span class="item-price">150</span>円</p>
  <a class="item-link" href="items/item01.html">くわしく見る</a>
</div>
```

このかたまりが、30個くり返されています。

名札の一覧

class	中身	使うステップ°
item	商品1つ分の箱	2
item-name	商品名	2
item-price	値段の数字だけ	3
item-category	カテゴリ	3
item-link	詳細ページへのリンク	4
item-stock	在庫（詳細ページだけ）	4

なぜ「150」と「円」が分けてある？

```
<span class="item-price">150</span>円
```

item-price の中には **数字だけ**。「円」は外に出してあります。

もし `<span class="item-price">150円</span>` だったら、
取れるのは **「150円」という文字**。

文字のままでは、大小をくられられません。

検証ツールで本物を見てみよう

1. サイトを開く

2. **右クリック** → **検証**（または F12）

3. 出てきたパネルが、そのページの HTML

`<div class="item">` が並んでいるのが見えます。

全部わからなくて大丈夫。

「**文字で書いてあるんだな**」が分かればOKです。

第2部

やってみる

ステップ0：まず相手を見る

コードを書く前に、**読ませたいページを自分の目で見る。**

<https://yuracode.github.io/isshop/>

- パン / 軽食 / 飲み物 / 文具 / 日用品
- ぜんぶで30商品

これが今日いちばん大事な順番です。

ステップ1：ページのタイトルを取る

やることは3つ。さっきの3つと同じです。

```
res = requests.get(BASE_URL)      # ① 取ってくる
res.encoding = "utf-8"
html = res.text

soup = BeautifulSoup(html, "html.parser")  # ② 分解する

print(soup.title.text)              # ③ 探す
```


. text を付けると中身だけ

書き方	出てくるもの
soup.title	<title>サイバー購買部 商品一覧</title>
soup.title.text	サイバー購買部 商品一覧

タグごとか、**中身だけ**かの違いです。

ほしいのは中身なので **. text** を付けます。

やってみよう

```
print(soup.h1.text)
```

`title` を `h1` に変えるだけ。

さっき説明した「title と h1 は違う」を、
自分の目で確かめてください。

ステップ2：商品名を全部ならべる

商品は30個。30回コピーするのは嫌ですね。

そこで `find_all` を使います。

命令	意味
<code>find</code>	最初の1つだけ持ってくる
<code>find_all</code>	全部まとめて持ってくる

箱を全部持ってくる

```
items = soup.find_all("div", class_="item")
print(len(items))
```

`div` のうち、`item` という名札が付いているものを全部

`len(...)` は個数を数える命令。 **30** と出れば成功です。

`class_` のうしろの `_` を忘れずに。

(Python では `class` が別の意味で使われているため)

箱の中から名前を取り出す

```
for item in items:  
    name = item.find("h2", class_="item-name")  
    print(name.text)
```

for item in items: は「items **ぜんぶに対して**、1つずつ」。取り出した1個が item です。

30個の箱を1つずつ順番に開けて、
その中から item-name の h2 を探して、中身を表示する

30行、ずらっと出ます。

ステップ3：値段も取る

名札を変えるだけです。

```
for item in items:
    name = item.find("h2", class_="item-name").text
    price = item.find("span", class_="item-price").text
    print(name, price)
    # print(price * 2)      ← 2個分の値段になる？
```

h2 → span、 item-name → item-price。 **新しいことは何もしていません。**

最後の行の # を外すと **`150150`**。まだ「文字」だからです。

文字の150 と 数の150 は別物

```
price_text = "150"    # ← 文字  
price      = 150      # ← 数
```

画面には同じに見えます。でもコンピュータには別物。

文字のままだと足し算も比較もできません。

```
price = int(price_text)  # 文字 → 数 に変換
```

一番安い商品を探す

考え方を日本語で言うと、こうです。

1. 「いままでで一番安い値段」を覚えておく箱を用意する
2. 上から順に見ていく
- 3. 覚えている値段より安かったら、覚えなおす**
4. 最後まで見たら、覚えているのが最安値

コードにするとこれだけ

```
cheapest_name = ""
cheapest_price = 99999

for item in items:
    name = item.find("h2", class_="item-name").text
    price = int(item.find("span", class_="item-price").text)
    if price < cheapest_price:
        cheapest_price = price
        cheapest_name = name
```

99999 は「最初はありませんな大きな数」を置いておくため。

できました

一番安いのは クリアファイル で 80 円です

30個の中から、一瞬で見つけかりました。

ここからは、取ったデータを使ってみます。

ステップ4：ほしいものだけ選ぶ

30個ぜんぶ出てきても、**多すぎて選べません。**

「120円以下だけ」「安い順に5つ」で見たいですね。

取ってから、選ぶ。 ここからがデータの使いかたです。

「もし～なら」で選ぶ

```
for item in items:
    name = item.find("h2", class_="item-name").text
    price = int(item.find("span", class_="item-price").text)
    if price <= 120:
        print(name, price)
```

if は「**もし～なら**」。条件に合うときだけ `print` します。

`<=` は「以下」。30個が **13個** にしぼれます。

安い順にならべる

```
prices = []

for item in items:
    name = item.find("h2", class_="item-name").text
    price = int(item.find("span", class_="item-price").text)
    prices.append((price, name))

prices.sort()
```

append は「**後ろに足す**」。sort() は「**小さい順にならべる**」。

(price, name) は **2つで1組**。sort() は組の**左から**くらのるので、**値段を先に**。
。

安いほうから5つ

```
for price, name in prices[:5]:  
    print(price, name)
```

```
80 クリアファイル  
90 使い捨てカイロ  
90 消しゴム  
100 シャープペンの芯  
100 炭酸水
```

`for price, name` と2つ書くと、組の左が `price`、右が `name` に入ります。

`prices[:5]` は「**最初の5つだけ**」。ステップ6でもまた出てきます。

ステップ5：カテゴリごとに数える

商品には、**カテゴリの名札**も付いています。

```
<p class="item-category">パン</p>
```

パンは平均いくら？ 飲み物は？ — **プログラムに数えさせます。**

同じ名札ごとに、まとめる

```
totals = {}  
counts = {}  
  
for item in items:  
    cat = item.find("p", class_="item-category").text  
    price = int(item.find("span", class_="item-price").text)  
    totals[cat] = totals.get(cat, 0) + price  
    counts[cat] = counts.get(cat, 0) + 1
```

`{}` は **辞書**。名前覚えておく入れものです。

「パン → 合計1090」「飲み物 → 合計710」のように覚えていきます。

平均を出す

```
for cat in totals:  
    print(cat, round(totals[cat] / counts[cat]))
```

パン 182 / 軽食 162 / 飲み物 118 / 文具 113 / 日用品 285

for cat in totals: は「辞書の**名札を1つずつ**」。round(...) は四捨五入。

日用品だけ、ずいぶん高いですね。

日用品は本当に高い？

安い順に見ると **90 / 110 / 150 / 180 / 200 / 980**。

980円の折りたたみ傘が1つ、平均を引き上げていました。

平均だけ見ると、まちがえます。**中身を見に行く。**

ページの外へ

リンクをたどってみる

ステップ6：リンクをたどる

一覧ページには **在庫が載っていません**。
在庫は、商品ごとの詳細ページにあります。

```
href = item.find("a", class_="item-link").get("href")
```

`.get("href")` で **属性の中身** を取り出します。

あとは、そのアドレスをまた `requests.get` するだけ。

やることはステップ1と同じです。

5つの商品を、順番に見に行く

```
for item in items[:5]:  
    ...  
    print(name, ":", stock)  
    time.sleep(1)
```

`items[:5]` は「最初の5商品だけ」。
`time.sleep(1)` は「**1秒待つ**」。

実行すると、**5秒くらい**待たされます。

ちょっと大事な話

使い方について

さっきの5秒

5個の商品を見に行くのに、**5秒**かかりましたね。

あれは、わざと **1秒ずつ待つように書いてあります**。

待たなければ、一瞬で終わったはずです。

なぜ、わざわざ待たせていると思いますか？

向こう側にもコンピュータがある

みんなが見ていたページは、

どこかのコンピュータが動いて返してくれています。

- 1回のアクセス → なんともない
- 待たずに30回、300回、3000回 → ?
- しかも、この教室には20人います

20人 × 3000回 が一斉に来たら、どうなるでしょう。

攻撃するつもりがなくても

結果として攻撃になってしまうことがあります。

だから今日は、

先生が用意した練習用のサイトを使いました。

よそのお店のサイトで練習したら、
それだけで迷惑になるからです。

判断するための3つの目安

見るもの	何が分かるか
利用規約	「自動で集めてはいけない」と書いてあることがある
robots.txt	機械向けの「入っていい／だめ」の案内板
間隔	1回ごとに1秒あける。急ぐ理由はたいていない

<https://例.com/robots.txt> で誰でも見られます。

同じコードが、道具にも迷惑にもなる

今日みんなが書いたコードは、

あと1文字変えるだけで、よそのサイトに向けられます。

分けているのは技術ではなく、**使い方**です。

できるようになった、というのはそういうことです。

発展

本物のページに触ってみる

ステップ7：本物のページに触る

ここまでは、**授業用に用意したきれいなページ**でした。

わざわざ `item-name` などの名札が付けてありました。

本物のページには、そんな親切はありません。

Wikipedia の記事で試してみましょう。

id = ページに1つだけの名札

```
print(wiki.find("h1", id="firstHeading").text)
```

	意味
class	同じ仲間が何個もある印（商品30個ぜんぶ item）
id	ページに1個しかない印

本物は、ほしいものだけ並べてくれない

```
for h in wiki.find_all("h2"):  
    print(h.text)
```

目次 ← これは記事の見出しじゃない！
名称の由来
歴史
...

いらないものが混ざるのが普通です。

取ったあとに「掃除」が要る

メロンパンとは、日本発祥の菓子パンの一種。……[1][2][3]

[1] は脚注の印で、`<sup>` というタグに入っています。

```
for sup in first.find_all("sup"):
    sup.decompose()      # そのタグをまるごと消す
```

本物のスクレイピングは、掃除が仕事の大半です。

そもそも入口が用意されていることも

Wikipedia には、

データを渡すための入口（API）があります。

HTML をむしるのではなく、そちらに聞く。

掃除の必要がない、きれいなデータが返ってきます。

相手が入口を用意しているなら、そっちを使うほうが速いし、相手にもやさしい。

名乗らないと、入れてもらえない

```
headers = {  
    "User-Agent": "授業用/1.0 (連絡先) requests"  
}
```

Wikipedia は「どこの誰が何のために来たのか」を求めています。

名乗らずにアクセスすると `403` (お断り) が返ります。

人間の世界と同じですね。

まとめ

きょうやったことは、たった3つのくり返しでした。

1. `requests.get(...)` で **取ってくる**
2. `BeautifulSoup(...)` で **分解する**
3. `find / find_all` で **探す**

そのあとは **選ぶ・ならべる・数える**。ふつうの計算です。

相手のサイトが変わっても、やることは同じ。
違うのは **「どこを探すか」** だけです。

おつかれさまでした

自分だったら、何を取り出してみたいですか？